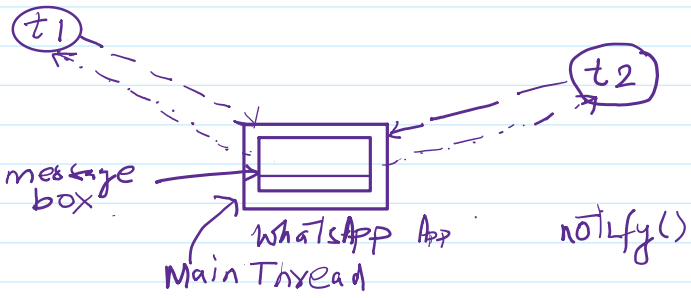
Inter thread communication

27 April 2026 18:38

Two threads can communicate with each other by using wait(), wait(long,int) notify(), notifyAll(), wait(long)



The thread which is expecting updation is responsible to call wait() method immediately, the thread will enter into waiting state.

The thread which is responsible to perform updatation. After performing updation it is responsible to call notify() method. Then waiting thread will get the notification and continue its execution with those updated items.

Why wait(), wait(long), wait(long,int), notify(), notifyAll() are not define in Thread class?

Ans:- wait(), wait(long), wait(long, int), notify() & notifyAll() present in Object class but not in Thread class because thread can call these methods on any Java object.

$t_1 \longrightarrow L(obj)$

obj.wait()

To call these 5 methods on any object, thread should be owner of that object i.e., the thread should have lock of that object, i.e., thread should be inside synchronized area.

Hence we can call any of these 5 methods only from synchronized area or block. Otherwise we will get RuntimeException saying IllegalMonitorStateException.

$t_1 \longrightarrow L(obj)$ $\qquad$ $t_2 \rightarrow L(obj)$

obj.wait() $\qquad$ obj.notify()

If a thread calls wait() method on any object, it immediately releases the lock of that particular object and entered into waiting state.

If a thread calls notify() method any object it releases the lock of the object.

Except wait(), wait(long), wait(long,int), notify(), notifyAll(), there is no method where thread release the lock.

Except wait(), wait(long), wait(long, int), notify(), notifyAll(), there is no method, where thread releases the lock.

Conclusions:

1. Two threads can communicate using wait(), notify & notifyAll() methods.
2. wait(), notify() & notifyAll() methods present in Object class of java.lang package not in Thread class.
3. wait(), notify() & notifyAll() methods can be called from synchronized block, otherwise we will get Runtime Exception: IllegalMonitorStateException.
4. If a thread calls wait() method on any object, immediately it releases the lock and it enters into waiting state.
5. If a thread call notify() method on any object it releases the lock but not immediately.

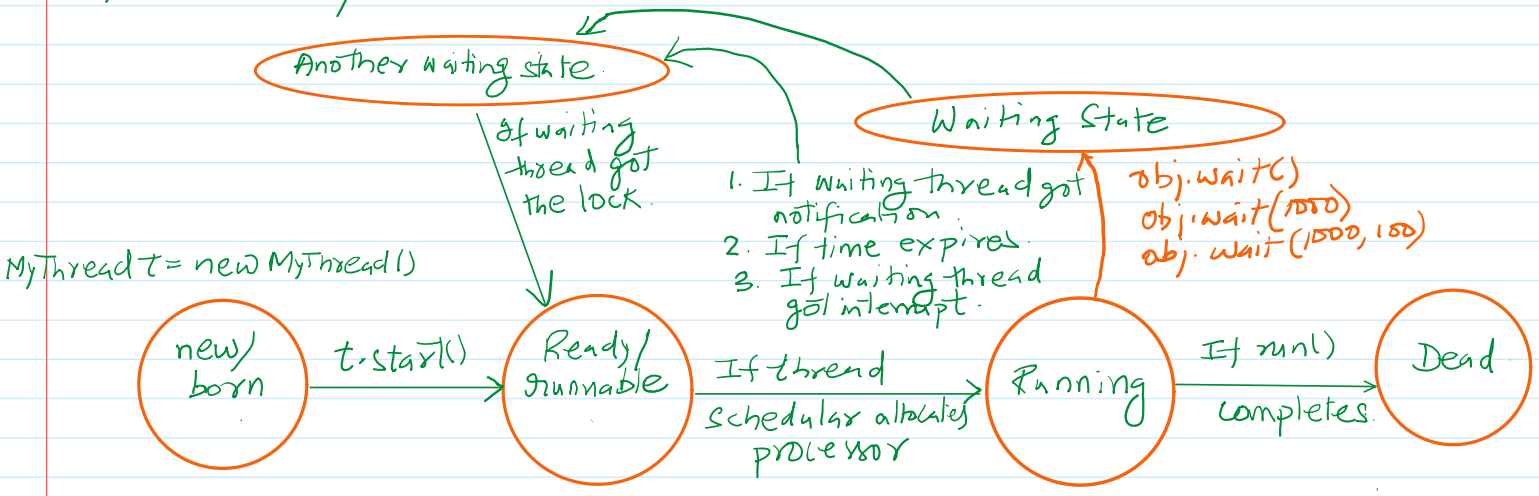
Def" of methods:

```
public final void wait() throws InterruptedException.
public final void wait(long ms) throws InterruptedException.
public final void wait(long ms, int ns) throws InterruptedException.
public final void notify()
public final void notifyAll()
```

All Methods	Instance Methods	Concrete Methods	Deprecated Methods

Modifier and Type	Method	Description
protected Object	clone()	Creates and returns a copy of this object.
boolean	equals(Object obj)	Indicates whether some other object is "equal to" this one.
protected void	finalize()	**Deprecated, for removal: This API element is subject to removal in a future version.** Finalization is deprecated and subject to removal in a future release.
final Class<?>	getClass()	Returns the runtime class of this Object.
int	hashCode()	Returns a hash code value for this object.
final void	notify()	Wakes up a single thread that is waiting on this object's monitor.
final void	notifyAll()	Wakes up all threads that are waiting on this object's monitor.
String	toString()	Returns a string representation of the object.
final void	wait()	Causes the current thread to wait until it is awakened, typically by being *notified* or *interrupted*.
final void	wait(long timeoutMillis)	Causes the current thread to wait until it is awakened, typically by being *notified* or *interrupted*, or until a certain amount of real time has elapsed.
final void	wait(long timeoutMillis, int nanos)	Causes the current thread to wait until it is awakened, typically by being *notified* or *interrupted*, or until a certain amount of real time has elapsed.

Every wait() method throws InterruptedException, which is a check exception. Hence, whenever we are using wait() method compulsorily we should handle this interrupted exception either by using try...catch or by throws keyword.



Thread life cycle using wait() & notify concept.